NEW CAPABILITIES · SIDE-BY-SIDE CHECKS

How the new features work and how to test them

The original staff management app is the baseline. This guide covers the newer PSS-AI additions, ML capabilities, student products, operating tools and controls.

Live-test prerequisite: confirm Cloudflare Pages publishes the Vite **dist** folder. The earlier read-only check of perfectstudyspace.pages.dev served raw JSX. Live migrations, function versions and secrets have not been verified.

Before you start

Verify migrations **001–072**, current **api / agent / cron** functions, correct backend project, required secrets and private storage. Enable one setting and its dependencies, then reload.

Use staging for invented payments, forced jobs, concurrency tests and fabricated history. A frontend preview can still point at the production backend.

Keep **pssai_agent_shadow = on** when first testing either AI chat. Off permits direct replies and allowed chat actions.

Local evidence: 109 unit tests passed; 29 server suites passed via Deno directly; production build passed. Server tests used simulated Supabase and AI.

Guide contents

01 Foundation and reliability
02 Study data and machine learning
03 Student portal and payments
04 AI assistants and communication
05 Growth, enquiries and seat resale
06 Self-service and branch operations
07 Learning and new student products
08 Controls to test alongside the features

Prepared 29 October 2026 · 75 capability/control rows · all 44 feature switches covered

A feature being present in source does not prove it is enabled, configured or validated on the live deployment. Read the result/limitation column alongside each test.

SECTION 01

Foundation and reliability

Follow the test with labelled test accounts. Check the expected result and any readiness limit before rollout.

Feature / setting	How it works	Method to test or check	Expected result / readiness limit
Action registry	Defines backend actions, allowed callers, risk levels and retry behavior.	Try an owner-only action as branch staff; try a scheduled action through the normal API.	Unauthorized requests are refused. Mainly a backend test.
Audit trail	Records actions, actors, results and related records in events.	Perform a labelled test action; inspect its row in Supabase → events.	Correct action/actor appears; secret fields are redacted.
Duplicate-submit protection	Stores responses for mutating requests carrying an idempotency key.	On staging, repeat the same payment request with the same key.	One payment is recorded; the retry receives the stored answer.
Scheduled jobs `pssai_cron`	Moves reminders and cleanup away from staff page loads.	Run the configured job helper; inspect the HTTP response and job history.	Jobs run without opening the app. The flag alone does not install or stop all schedules.
Nightly integrity report	Detects inconsistent memberships, payments, sessions and desks.	Run cron_integrity_report; open Dashboard's data-check card.	A fresh report shows actual findings or a clean result.

SECTION 02

Study data and machine learning

Follow the test with labelled test accounts. Check the expected result and any readiness limit before rollout.

Feature / setting	How it works	Method to test or check	Expected result / readiness limit
Focus pulse `pssai_focus_pulse`	Saves an optional 1–5 focus rating after a member's checkout.	Staff → Bookings → check out member → select rating → inspect portal Report.	One rating per booking. Partial: student self-checkout cannot collect ratings yet.
Focus Fingerprint	Groups members by study habits using locally trained clustering.	Export/train ML → inspect cluster report → publish → inspect fingerprints.	Needs history and owner-approved labels. No dedicated staff fingerprint screen.
Churn prediction	Estimates whether a member is likely not to renew.	Train on real history → compare with seven-day reminder baseline → publish.	student_scores contains churn scores/reasons. Real accuracy still needs measurement.
Engagement scoring	Combines attendance, quota use, focus and churn information.	Publish ML results; inspect student_scores filtered to engagement.	Scores should match member activity. Dedicated ranking UI is not wired.
Anomaly detection `pssai_anomaly`	Flags unusual discounts, waivers, attendance edits and similar records.	Publish results → Owner → Flags → mark It's fine or Problem → reload.	Verdict persists. Needs real-data publication and about two weeks of false-alarm review.
Cash-flow forecast	Estimates collections from renewal likelihood, dues and other income.	Publish → inspect forecasts; ask analyst for the dated forecast.	Forecast includes a range. Needs history; no dedicated forecast chart screen.

ML workflow: export → train locally → review reports/labels → publish. SQL switches do not train these models. Ollama is not integrated; the generative assistants use Anthropic’s API.

SECTION 03

Student portal and payments

Follow the test with labelled test accounts. Check the expected result and any readiness limit before rollout.

Feature / setting	How it works	Method to test or check	Expected result / readiness limit
Portal setup and login `pssai_portal`	Staff creates a one-time setup code; the student chooses a PIN and gets a separate session.	Student profile → Portal setup code → /me/setup → create PIN → /me/login.	Code cannot be reused. Requires STUDENT_JWT_SECRET and schema setup.
Student account isolation `pssai_portal`	Derives student identity from the portal token for every request.	Log in as two test students in separate browsers; attempt cross-account access on staging.	Each sees only their own account, payments and study records.
Membership home `pssai_portal`	Shows plan, dues, expiry, locker and Food Pass information.	Open /me; compare every value with the staff profile.	Values agree with staff records.
Study report `pssai_portal`	Summarizes sessions, daily study time and available focus ratings.	Open /me/report; choose dates and compare against bookings.	Hours and dates match the underlying sessions.
Receipts `pssai_portal`	Shows payments, refunds and payment-proof decisions.	Complete a staging payment/refund; open /me/receipts.	Correct category, amount, date and status appear.
Inbox and notices `pssai_portal`	Stores account notifications for the student.	Trigger a payment/request decision → Inbox → mark read.	Correct student receives the notice; unread count changes.
UPI payments `pssai_upi_pay`	Creates a priced request; owner checks the bank and confirms the proof.	/me/pay → submit proof → Owner Approvals → confirm/reject → inspect receipts and Revenue.	No credit before confirmation; one confirmation runs one billing action. Requires UPI settings/storage.
Screenshot reading `pssai_proof_reading`	Claude reads amount, UTR and payee; code compares with the request.	Submit clear, wrong-amount and unclear screenshots; inspect warnings.	AI never approves payment. Requires AI key/budget. Flag controls automatic reads.
Approval queue	Collects sensitive student/agent requests for owner decisions.	Request a long hold/refund → Approvals → approve/deny.	Request waits for owner; some approved requests still require manual fulfillment.

Feature / setting	How it works	Method to test or check	Expected result / readiness limit
Self-service holds `pssai_portal`	Eligible temporary members can start a short hold within the monthly limit.	/me/hold → request short hold → inspect membership → resume.	Eligible hold starts; longer, permanent or second holds enter Approvals.
Plan, refund and privacy requests `pssai_portal`	Records student requests and the owner's decision.	Submit each type → approve/deny → inspect notice.	Decision works. Approval does not automatically perform refunds, deletion or data export.
Signup payment `pssai_auto_approve`	Adds plan, PIN and payment to signup; owner confirmation completes registration.	Branch signup link → details/payment → owner confirms → inspect membership/login.	One registration/payment. Blocked until same-domain signup routing is fixed.

SECTION 04

AI assistants and communication

Follow the test with labelled test accounts. Check the expected result and any readiness limit before rollout.

Feature / setting	How it works	Method to test or check	Expected result / readiness limit
Owner analyst `pssai_analyst`	Claude calls approved read-only business tools to answer owner questions.	Ask yesterday's branch collections; compare with Revenue. Ask it to grant a refund.	Figures match; it cannot execute the refund. Requires live AI setup and evaluation.
Morning owner brief `pssai_analyst`	Produces a daily collections, issues and priorities summary.	Run owner_brief; inspect Dashboard and agent_runs.	A dated brief appears; successful run shows model calls and tokens.
Student concierge `pssai_concierge`	Answers own-account questions; allowed actions depend on review mode.	Keep agent_shadow on → /me/chat → ask dues/hold questions → inspect Chat replies.	Draft waits for staff and matches account facts. Needs live accuracy evaluation.
Chat reply review	Staff can edit, send or discard assistant drafts.	Edit and send one draft; discard another; inspect student chat.	Only sent replies become visible to the student.
Collections `pssai_collections`	Ranks expiring/unpaid members and drafts reminders.	Run collections → Send list → compare reasons/dues → mark sent/skipped.	Correct candidates, no duplicate daily reminders. WhatsApp sending stays manual.
Welcome and report drafts `pssai_ai_messages`	Rewrites supplied facts into messages without calculating money.	Generate a welcome/study-report draft; compare numbers, dates and links.	Facts stay unchanged; staff edits before sending. Requires AI key/budget.
Exception desk `pssai_exception_desk`	Adds an AI recommendation to a pending approval.	Create request → Approvals → request AI suggestion.	Suggestion appears; request remains pending until owner decides.

> **AI check:** inspect the new agent_runs row. A successful live test has status = ok, model_calls > 0 and populated usage. HTTP 200 can also mean skipped; read the response content.

SECTION 05

Growth, enquiries and seat resale

Follow the test with labelled test accounts. Check the expected result and any readiness limit before rollout.

Feature / setting	How it works	Method to test or check	Expected result / readiness limit
Public enquiry assistant `pssai_enquiry_chat`	Answers visitors using branch/plan facts and records their enquiry.	Logged out → /ask → ask plans → Enquiries → Website chat.	Enquiry/messages persist and drafts can be reviewed. Needs AI and shadow evaluation.
Lead scoring `pssai_lead_tools`	Scores enquiries from activity and follow-up facts.	Create varied test enquiries → run cron_score_enquiries → inspect reasons.	Scores reflect the supplied facts. Basic scoring does not require AI.
Duplicate suggestions `pssai_lead_tools`	AI reviews plausible duplicate pairs; staff decides whether to merge.	Create similar-name/different-phone enquiries → run enquiry_dedupe.	Suggestions appear; no automatic merge. Needs qualifying pairs and AI.
Campaign referral links `pssai_referral_links`	Attributes signup, chat and join activity to campaign links.	Growth → Referral links → create link → signup/register → run conversion job.	Counts increase once. Signup routing currently blocks the complete flow.
Friend referrals and cashbacks `pssai_portal_r2` `pssai_referral_links`	A student's referral can earn a reward that waits for owner approval.	More → Bring a friend → register referred student → inspect reward approval.	Cashback follows approval. WhatsApp/Other source validation needs alignment.
Portal waitlist/cashback pages `pssai_portal_r2`	Exposes the student's own waitlist and cashback records.	More → waitlist/cashbacks → join/leave → compare staff records.	Own records/statuses match; related features have separate switches.
Released cabin hours `pssai_hour_release`	Member releases hours; walk-in resale generates a 50% credit.	Release window → sell it → Growth → Seat resales/cashback; try duplicate sale.	One sale and correct credit. Test original-member arrival and fallback.
Predicted unused hours `pssai_overbooking`	Uses history to suggest cabin windows likely unused.	Run predictions → inspect offers → sell on staging → simulate member arrival.	Needs history and controlled collision testing. Shared seat issue remains.

Feature / setting	How it works	Method to test or check	Expected result / readiness limit
Daily walk-in pricing `pssai_yield_pricing`	Adjusts base prices within configured limits using occupancy/exams.	Set +10% in Growth → compare quote; change price while old form is open.	Correct price charged; stale quote rejected. Automation needs history.
Exam radar `pssai_exam_radar`	Reads public sources and proposes exam notices for review.	Run radar → Growth → Exam calendar → compare source → confirm/dismiss.	Only confirmed notices affect downstream behavior. Needs live source/AI evaluation.
Win-back `pssai_winback`	Finds suitable former members and drafts a limited return offer.	Run job → Send list → inspect eligibility, discount and validity.	Valid candidates/drafts; sending and discount application remain manual.
Next-exam plan `pssai_next_exam`	Uses the exam family/ladder to suggest a follow-on plan.	Use supported course → /me/next → compare next step and price.	Appropriate plan; unsupported courses may have no offer.
Pay-later advice `pssai_paylater_advice`	Gives explainable risk advice from payment/renewal history.	Run scoring → compare good and unpaid histories.	Reasons make sense; advice does not automatically approve credit.
Waitlist offers `pssai_waitlist_offers`	Offers a free seat to an eligible waiting student for 24 hours.	Free staging desk → run job → accept in portal → inspect Approvals.	One offer holds availability; staff completes seat/plan assignment.

SECTION 06

Self-service and branch operations

Follow the test with labelled test accounts. Check the expected result and any readiness limit before rollout.

Feature / setting	How it works	Method to test or check	Expected result / readiness limit
QR/location check-in `pssai_self_checkin`	Validates the poster key and location within 100 m, then runs member check-in.	Configure location/poster → scan inside/outside radius → rotate poster → retry old link.	Valid scan works; outside/old key fails. Cross-branch permanent-member seating needs fixing.
Self-checkout `pssai_self_checkin`	Runs checkout, carrying overtime to renewal and applicable food as dues.	Student Home → checkout → compare session, overtime and food balances.	Session closes once with the correct amounts.
Seat matching `pssai_seat_matching`	Ranks free desks using favorites and tags.	Tag desks → save preferences → check in → occupy favorite and retry.	Best eligible desk selected. Shared cross-branch seat bug remains.
Free-seat counts `pssai_live_seats`	Shows branch availability without student identities.	Compare portal counts before/after check-in/out; refresh.	Counts match records. This flag alone is not a streaming seat map.
Session reminder `pssai_session_reminders`	Creates an in-app notice shortly before booked end.	Use staging session ending within ten minutes → run job twice.	One reminder; paused sessions excluded. Requires schedule/portal.
Forgotten-session closure `pssai_session_watch`	Closes overdue sessions at booked end/break start and flags review.	On staging create overdue/future sessions → run job → inspect bookings/flags/notices.	Only eligible sessions close; job adds no overtime. Manual run affects eligible branches.
Staff correction flags `pssai_session_watch`	Flags qualifying staff checkouts backdated by over ten minutes.	Make labelled staging correction → run session-watch job → inspect Flags.	Correction appears for review.
Student food ordering `pssai_food_orders`	Queues orders; delivery creates the normal food bill.	Order → Accept → Ready → Deliver → inspect bill/pass/tab; test cancellation/price change.	One delivered order creates one bill. Hold concurrent Food Pass rollout until debit race is fixed.
Stock deduction `pssai_food_stock`	Deducts linked inventory when menu items are billed.	Link menu item → sell → inspect inventory quantity/log.	Correct units deducted; failed writes need reconciliation.

Feature / setting	How it works	Method to test or check	Expected result / readiness limit
Demand forecast `pssai_food_stock`	Estimates food demand from recent sales patterns.	Food orders → Stock & suppliers → compare forecast with history.	Useful with history; new items may have weak/empty forecasts.
Supplier drafts `pssai_food_stock`	Drafts replenishment; staff sends and records actual received stock.	Run supplier job → open draft → mark sent → receive actual quantity.	Actual received stock added once. Supplier message stays manual.
Problem tickets `pssai_tickets`	Routes issues to branch staff and prioritizes urgent reports.	Submit labelled TEST issue → Problems → resolve → inspect student notice.	Correct priority and resolution notification.
Photo-proof tasks `pssai_photo_tasks`	Requires a photo before completion of configured tasks.	Require photo → try without → upload → complete → owner views.	Missing photo blocks completion. Existing tasks need the requirement applied.
Staff live refresh `pssai_realtime`	Broadcasts a signal so other tabs re-fetch authorized data.	Open two tabs → update in one → observe the other.	Refresh without manual reload; requires working websocket setup.

SECTION 07

Learning and new student products

Follow the test with labelled test accounts. Check the expected result and any readiness limit before rollout.

Feature / setting	How it works	Method to test or check	Expected result / readiness limit
Policy learner `pssai_policy_learner`	Proposes rules from owner decisions; only signed eligible hold rules can apply automatically.	Qualifying staging decisions → run proposals → sign rule → matching hold → run application job.	Unsigned rules do nothing; only permitted holds auto-approve. Needs decision history.
What-if simulation `pssai_digital_twin`	Replays history under proposed limits without changing real policy.	Learning → What if → change limits → compare records/settings before/after.	Returns differences; real accounts remain unchanged.
Experiments `pssai_experiments`	Assigns stable variants, measures conversion and lets owner keep a winner.	Create → expose test students → revisit → inspect result/Keep/Drop.	Variants stay stable. Meaningful winner needs sufficient real samples.
Space-use analysis `pssai_space_yield`	Builds occupancy heatmaps and suggestions from usage/waitlist data.	Learning → Space use → compare branch/hour cells with bookings.	Needs history and correct seating data.
Exam coach `pssai_portal_r4`	Compares study activity with plan hours and a confirmed exam.	More → Exam coach → compare course, countdown and last 14 days.	Accurate figures/tips; supported exam data required.
Focus profile `pssai_portal_r4`	Finds stronger study windows from focus ratings.	Collect five staff-assisted ratings → More → When you focus best.	Window/day averages appear. Student-only rating input is unfinished.
Monthly Wrapped `pssai_portal_r4`	Summarizes hours, attendance, streaks, preferred desk and focus.	More → Your month → choose month → compare sessions.	Summary matches source records.
Study buddy `pssai_study_buddy`	Matches consenting students by branch, exam family and study window.	Two compatible students opt in → run matching → accept one side, then both.	Identity appears after mutual acceptance; phones stay hidden.
Staff-module retirement `pssai_retire_staff_ops`	Retires attendance, cover board, End Session and staff messaging while retaining tasks/notices.	Inspect usage; test retirement and rollback on staging accounts.	Wait for the 30-day operational gate. Duty-only permissions remain unfinished.

Feature / setting	How it works	Method to test or check	Expected result / readiness limit
Study certificates `pssai_certificates`	Signs completed study totals and offers print/QR/public verification.	Issue → scan QR logged out → compare → revoke → reopen.	Verifies signed facts; revoked result hides details. Needs stable certificate secret.
Weekly family summary `pssai_parent_digest`	Provides a revocable link to weekly study snapshots.	Create link → generate staging snapshot → open logged out → replace/revoke.	Study facts only; old link stops working. Requires snapshot schedule.
Every-branch overview `pssai_owner_overview`	Aggregates collections, dues, seats, approvals, problems and orders.	Compare rows/totals with branch screens; try staff login.	Totals agree; owner/admin only. Collections are gross transactions.

SECTION 08

Controls to test alongside the features

Follow the test with labelled test accounts. Check the expected result and any readiness limit before rollout.

Feature / setting	How it works	Method to test or check	Expected result / readiness limit
Chat review `pssai_agent_shadow`	on means staff reviews AI drafts; off permits direct replies/actions within allowed rules.	Keep on; verify concierge/enquiry drafts wait for staff.	One shared setting affects both chats. Start testing with on.
Monthly AI budget `pssai_ai_budget_usd`	Application-level monthly spend control; numeric USD value.	On staging set 0 → try AI call → inspect reason → restore chosen budget.	Call is skipped/refused for budget. An 'off' string falls back to defaults, not zero.
Public image-read limit `pssai_public_ai_daily`	Limits automatic public-triggered ID/payment image reads.	On staging set 0 → submit image → inspect saved submission/read status.	Submission remains; automatic AI reading is skipped. Not a global owner-read limit.
Public enquiry AI limit `pssai_enquiry_ai_daily`	Limits daily public enquiry AI replies.	On staging set 0 → submit enquiry message → inspect staff queue.	Message retained for handling; no new AI reply.
Extra discount confirmation `pssai_discount_confirm_over`	Sets a rupee threshold for extra confirmation.	Try an above-threshold staging discount; cancel confirmation.	No change after cancellation. This is a numeric control, not a feature toggle.

Highest-priority remaining work

Correct frontend publishing and same-domain signup routing; fix concurrent Food Pass debits and cross-branch permanent-member desk claiming; align referral-source validation; add student focus input; complete duty-specific access. Verify real database integration, live AI accuracy and operational pilot gates.

Record for each test: feature, deploy/commit, backend project, actor/branch, test IDs, expected/actual result, evidence and pass/fail. Switching a flag off does not undo payments, completed sessions or other effects.

Source references: docs/PSSAI-FEATURE-TESTING.md · docs/ops/cron.sql · docs/ops/ai-test-setup.sql · docs/plans/PSS-AI-execution-plan.md